SpeakIT: Enterprise Engineering & Architecture Deep-Dive

An exhaustive architectural handbook, system design review, and technical interview guide.

TABLE OF CONTENTS

---

1. PROJECT OVERVIEW

What Was Built

SpeakIT is a production-grade, full-stack Text-to-Speech (TTS) SaaS platform. It converts user-provided text into highly realistic, human-quality audio by orchestrating requests to AWS Polly. The platform implements robust, plan-based rate limiting (Free vs. Pro tiers), secure stateless authentication using custom JWT handling, a comprehensive marketing and analytics suite, and a highly optimized streaming infrastructure designed for minimal latency.

Live URLs:

- Frontend: `mohitur-speakit.vercel.app` (Angular SPA on Vercel)
- Backend: `text-to-speech-java-backend.onrender.com` (Spring Boot on Render)
- GitHub: `github.com/Mohitur669/speakit`

Core Business Purpose

SpeakIT democratizes access to studio-quality voice generation for content creators, developers, educators, and businesses. It eliminates the prohibitive costs, complex studio setups, and slow turnaround times traditionally associated with voice actors by providing an instant, API-driven, and highly polished user interface overlaid on top of advanced neural speech engines.

Problem Being Solved

Legacy TTS systems are notoriously robotic, lacking natural cadence and emotional inflection. Professional voiceover work, on the other hand, is expensive, difficult to iterate upon, and impossible to automate for dynamic content (such as programmatic video generation, news readers, or accessibility tools). SpeakIT bridges this gap by abstracting the complexity of AWS Polly into a frictionless, monetizable product with a seamless developer and user experience.

Product Vision

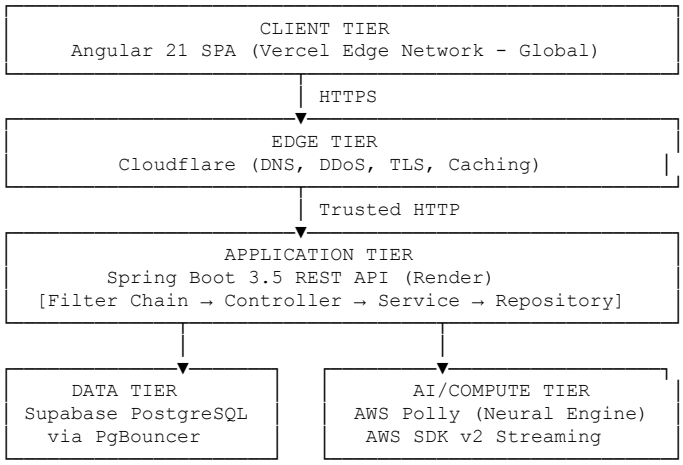
To become the definitive infrastructure layer for the next generation of auditory experiences. Future iterations aim to support:

- Open-core API integrations
- Custom voice cloning
- Team collaboration features
- Enterprise-grade SLA-backed API access
- Audiobook generation with async job queuing

2. HIGH-LEVEL ARCHITECTURE

SpeakIT utilizes a decoupled, modern cloud architecture designed for high availability, security, and extremely low-latency audio delivery.

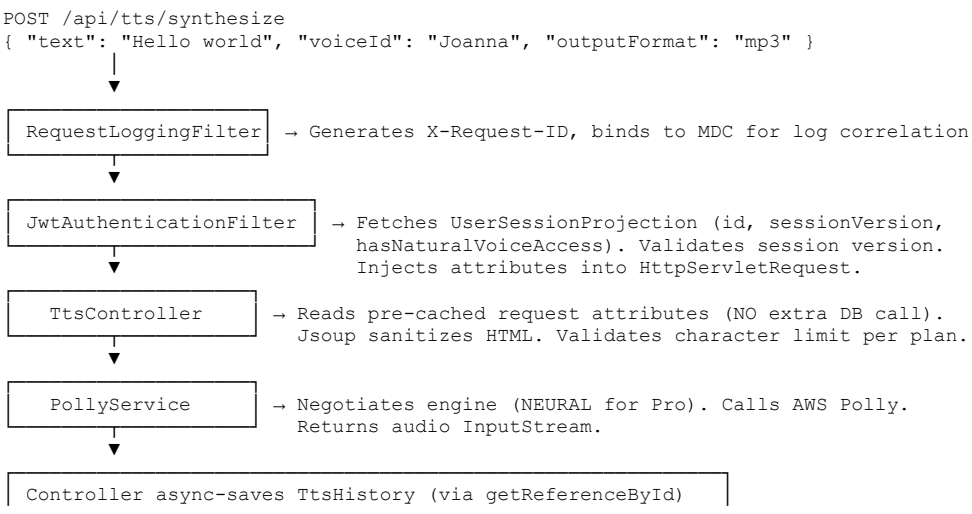
Architecture Layers



System Flow (Step by Step)

1. **Client Tier:** The user interacts with the Angular 21 Single Page Application (SPA). This is statically compiled and served globally via Vercel's Edge Network for near-zero TTFB (Time to First Byte).
2. **Edge Tier:** Cloudflare manages DNS and edge routing, enforcing strict SSL/TLS policies, caching static assets, and providing initial DDoS protection before traffic ever reaches the origin servers.
3. **Application Tier:** The Spring Boot 3.5 REST API (hosted on Render) receives the request. It processes CORS preflight checks, validates the JWT, enforces Bucket4j rate limits, and executes core business logic.
4. **Data Tier:** Supabase PostgreSQL acts as the source of truth for user identities, subscription states, and conversion history. All database traffic is routed through an IPv4 Session Pooler (PgBouncer) provided by Supabase to prevent connection exhaustion in containerized environments.
5. **AI/Compute Tier:** The backend establishes a secure, authenticated connection to AWS using the AWS SDK v2, invoking either the Polly Neural or Standard engine to generate the raw audio stream.

Request Lifecycle — The Synthesis Hot-Path



```
│ Streams audio/mpeg directly to HTTP response OutputStream  │
└────────────────────────────────────────────────────────────┘
```

Why this matters in an interview: The hot-path is designed so that a single TTS request touches the database exactly **once** (the JWT projection), and records history **asynchronously** without blocking the audio stream. This is deliberate — it keeps latency as low as possible.

Interview Questions — Architecture

Q: Why did you split the frontend and backend into separate hosted services rather than serving the Angular app from Spring Boot?

A: This is called a "decoupled deployment" pattern and it has three major benefits:

1. **Independent scaling:** The frontend is static HTML/JS/CSS. Serving it from Vercel's global CDN means it scales infinitely with zero server cost, and users worldwide get sub-10ms TTFB from the nearest edge node. Spring Boot doesn't need to handle any static file serving.
2. **Separation of concerns at infra level:** The Angular app can be deployed 10 times a day (CSS fixes, copy changes) without ever touching the backend. Backend deployments (schema migrations, Java upgrades) don't force a frontend rebuild.
3. **Independent tech evolution:** The frontend can be rewritten in Vue or SvelteKit tomorrow without touching a single line of Java. This is the correct architectural pattern for any SaaS that expects to grow a team.

The tradeoff is CORS configuration, which we handle via `application.properties` with environment variable overrides so the allowed origins differ between dev, staging, and prod without recompilation.

Q: What is TTFB and why does it matter for a TTS application?

A: TTFB (Time to First Byte) is the time between the browser sending an HTTP request and receiving the first byte of the response. For the Angular SPA, low TTFB means the user sees the UI shell faster. Vercel's edge network serves the compiled HTML from a CDN node closest to the user — typically under 30ms globally — vs. serving from a single Render server in a US region which could be 200-400ms for Indian users.

For the actual audio synthesis (`/synthesize`), TTFB represents how quickly the audio starts playing. We optimize this through streaming: we pipe the AWS Polly `InputStream` directly to the `HttpServletResponse` `OutputStream`, so the browser receives the first audio bytes while Polly is still generating the rest.

Q: Why Render for the backend instead of AWS EC2 or Railway?

A: At the startup/MVP stage, the primary concern is operational velocity. Render provides:

- Zero-config Docker container deployment from a GitHub push
- Automatic HTTPS with managed TLS certificates
- Environment variable management without an AWS IAM learning curve
- Free tier for initial validation (with the keep-alive workaround for spin-down behavior)

The tradeoff is vendor lock-in and less infrastructure control. When we need auto-scaling groups, VPC peering with our RDS instance, or custom networking, we migrate to ECS (Fargate) or EKS. The Spring Boot app is containerized (Docker Hub: `mohitur/speakit:backend`), so the migration is a configuration change, not a code change.

---

3. FRONTEND ARCHITECTURE

The frontend is built with **Angular 21**, strictly utilizing **Standalone Components** and **Signals**. It abandons legacy Angular patterns in favor of modern, high-performance web development standards.

Feature Modularization & Folder Structure

```
src/
├── app/
│   ├── core/          → Singletons: AuthService, TtsService, Interceptors, Guards, LoggerService
│   ├── shared/        → Reusable presentational components: Navbar, Footer, Toast
│   ├── features/
│   │   ├── auth/      → Login, Register, Forgot Password (lazy-loaded)
│   │   ├── tts/       → TTS Studio (lazy-loaded)
│   │   └── marketing/ → Landing page, Pricing (lazy-loaded)
```

```
│   └── environments/  → Base config files (runtime values injected via window.__env)
├── public/
│   └── runtime-env.js → Generated at build/start time; injects API URL into window.__env
└── scripts/
    └── set-env.js     → Node script run via prestart/prebuild npm hooks
```

Core Design Decisions

1. Signals over RxJS for Component State

We use Angular Signals (`signal`, `computed`, `effect`) for all component-level state management — character counts, loading flags, form error states. RxJS is strictly reserved for asynchronous operations that cross network boundaries (i.e., `HttpClient` calls).

Why Signals?

- **Fine-grained change detection:** Angular's default `ChangeDetectionStrategy.Default` re-evaluates the entire component tree on every event. Signals are granular — only the DOM nodes that depend on a changed signal re-render. This eliminates unnecessary view traversals.
- **No subscription management:** With RxJS, forgetting to `unsubscribe()` or use `takeUntilDestroyed()` causes memory leaks. Signals are garbage-collected automatically.
- **Glitch-free:** Signals use a push-pull model that guarantees consistency; you can never read a computed value that is in an intermediate, inconsistent state (a "glitch").

Tradeoff: Signals don't replace RxJS for complex event streams (e.g., debounced search inputs, WebSocket streams, retry logic). Those remain RxJS territory.

2. Centralized LoggerService

Developers are strictly forbidden from using `console.log`, `console.error`, or `console.warn` directly.

Why?

- The `LoggerService` wraps all console output and checks `LOG_LEVEL` from the runtime environment. In production, it silences `DEBUG` and `INFO` noise.
- It contains a recursive `sanitize()` method that redacts keys like `token`, `password`, and `jwt` from any object before logging. This prevents catastrophic PII leaks to the browser console — a common, underrated SPA vulnerability.

Example:

```
// BAD (forbidden)
console.log("User logged in:", { user: { id: 1, token: "eyJ..." } });

// GOOD
this.logger.debug("User logged in:", { user: { id: 1, token: "eyJ..." } });
// Output in dev:  User logged in: { user: { id: 1, token: '[REDACTED]' } }
// Output in prod: (silenced — level is WARN+)
```

3. Cross-Tab Session Sync (BroadcastChannel)

Implemented in `AuthService` using the browser `BroadcastChannel` API.

Why? If a user has Tab A (TTS Studio) and Tab B (Account Settings) open, and logs out in Tab B, Tab A still has a valid JWT in memory and localStorage. Without this, Tab A could continue making authenticated API calls on a session the user thought they ended.

How it works:

```
Tab B: user clicks "Logout"
  → AuthService clears localStorage, calls /api/auth/logout
  → BroadcastChannel.postMessage({ type: 'LOGOUT' })

Tab A: BroadcastChannel listener fires
  → Wipes localStorage
  → Navigates to /login
  → User sees: "You've been logged out from another tab"
```

This guarantees a consistent security state across all browser windows, zero server-side polling required.

4. Build-Once, Deploy-Anywhere (Runtime Environment Injection)

Instead of baking the API URL into `environment.prod.ts` at compile time, a Node.js script runs via `prebuild`/`prestart` hooks. It writes a `window.__env` object to `public/runtime-env.js`, which is loaded by `index.html` before Angular bootstraps.

Why this is critical:

```
WITHOUT this pattern:
  Build for Dev  → angular.json points to dev API
  Build for Prod → angular.json points to prod API
  □ Two different compiled artifacts. The artifact tested in Staging ≠ Prod artifact.

WITH this pattern:
  One build. One Docker image.
  Dev environment  → injects dev API URL at runtime via env vars
  Prod environment → injects prod API URL at runtime via env vars
  □ The exact artifact tested in Staging IS the Prod artifact.
```

This is standard CI/CD hygiene at companies like Netflix and Uber and is required for Docker image promotion workflows.

Interview Questions — Frontend

Q: Why Angular over React for this project?

A: The decision was based on the existing skill set and the nature of a production-grade SaaS. Angular's opinionated structure (enforced module/component separation, built-in DI, official router, official HttpClient) reduces architectural decision fatigue. In a team setting, every Angular project looks structurally the same, which accelerates onboarding.

React is more flexible but that flexibility becomes a liability at scale — you need to pick and coordinate: state (Zustand/Redux/Jotai?), routing (React Router/TanStack?), data fetching (React Query/SWR?). Angular makes those choices for you.

The tradeoff: Angular has a steeper initial learning curve and a larger bundle size baseline. Angular 21's standalone components and Signals significantly narrow the gap with React's modern mental model.

Q: What is an Angular Guard and where did you use it?

A: A Route Guard is middleware that executes before a route is activated. We implement `CanActivateFn` guards:

- **`AuthGuard`:** Checks if the user has a valid JWT in localStorage. If not, redirects to `/login`. Applied to all `/tts` routes.
- **`GuestGuard`:** The inverse — prevents authenticated users from visiting `/login` or `/register`. Redirects them to `/tts/studio`. Prevents the awkward UX of a logged-in user seeing the login form.
- **`ProGuard`:** Checks `hasNaturalVoiceAccess` from the decoded JWT payload. If a Free user tries to access Neural Voice features directly via URL, they get redirected to the pricing page.

Q: What is an HTTP Interceptor and what do you use it for?

A: An Angular HTTP Interceptor is a middleware layer in the `HttpClient` pipeline that can read and modify every outgoing request and incoming response without touching individual service methods.

We use two:

1. **`AuthInterceptor`:** Reads the JWT from localStorage and attaches it as `Authorization: Bearer <token>` to every outgoing request automatically. Without this, every service method would need to manually add the header.

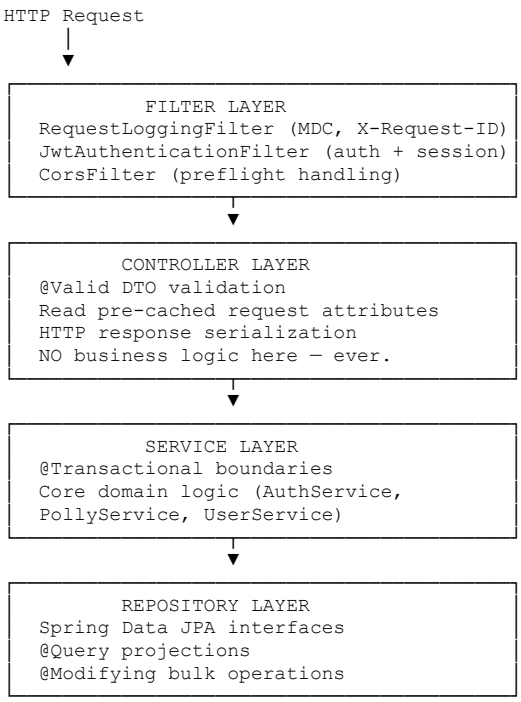
2. **`ErrorInterceptor`:** Catches 401 responses globally. If the server returns 401 (expired or invalidated session version), the interceptor automatically calls `AuthService.logout()`, clears state, and redirects to `/login`. This prevents the user from being stuck in a broken authenticated state.

---

4. BACKEND ARCHITECTURE

The backend is engineered using **Spring Boot 3.5** and **Java 21**, enforcing a strict layered architecture to separate concerns, maximize testability, and ensure maintainability.

Layered Architecture



Core Design Decisions

> **Note:** Rate limiting is a major cross-cutting concern. All rate limiting architecture — AOP implementation, Bucket4j configuration, protection zones, thread safety, and the Redis migration path — is covered in full in **Section 7: Rate Limiting & Traffic Governance**.

1. Disabling Open-Session-In-View (OSIV)

We explicitly set `spring.jpa.open-in-view=false` in `application.properties`.

What OSIV does by default: Spring Boot, by default, keeps the Hibernate `Session` (and therefore the underlying JDBC connection) open for the entire HTTP request lifecycle — including the time spent serializing the response to JSON and sending it over the network.

Why this is catastrophic at scale:

```
Request arrives → Connection checked out from HikariCP pool
  → Service executes DB queries (fast, <10ms)
  → Jackson serializes response to JSON (fast)
  → Network transmits response to client (SLOW — 100-500ms for large audio metadata)
Connection returned to pool ← ONLY HERE

With pool size = 10:
10 concurrent slow-network clients = ALL connections exhausted
New requests: "HikariCP connection timeout" → HTTP 500
```

With OSIV disabled, connections are returned to the pool the moment the `@Transactional` service method completes — well before the response starts serializing.

3. Strict DTO Pattern

JPA Entities are **never** returned to the frontend. All responses are mapped to DTOs (Data Transfer Objects).

Why?

1. **Security:** An `@Entity User` object contains `passwordHash`, `sessionVersion`, internal `id` sequences. Accidentally returning the entity exposes all of this. A DTO exposes exactly: `{ id, email, plan, hasNaturalVoiceAccess }`.
2. **API Contract Stability:** If you return entities directly, renaming a database column (`voice_id` → `voice_identifier`) breaks all API consumers immediately. DTOs create an abstraction layer. The DB can change; the DTO stays the same.

3. **Serialization Safety:** Hibernate entities with `@OneToMany` / `@ManyToOne` relationships are bidirectional. Jackson will attempt `User → [History] → User → [History] → ...` resulting in a `StackOverflowError`. DTOs are plain POJOs with no circular references.

Interview Questions — Backend

Q: Explain the Spring Security filter chain and where your custom filters fit.

A: Spring Security implements a `FilterChainProxy` — a chain of servlet filters that every HTTP request passes through sequentially. Each filter has a specific responsibility:

Standard filters (in order): `SecurityContextPersistenceFilter` → `UsernamePasswordAuthenticationFilter` → `ExceptionTranslationFilter` → `FilterSecurityInterceptor`.

We add custom filters before `UsernamePasswordAuthenticationFilter`:

1. **`RequestLoggingFilter`** — fires first. Generates the `X-Request-ID`, binds it to SLF4J MDC. Every subsequent log statement in this thread automatically includes this ID.
2. **`JwtAuthenticationFilter`** — fires second. Extracts the `Authorization: Bearer` token, decodes it, validates the `sessionVersion` against the DB projection, and sets the `Authentication` object in `SecurityContextHolder`. All downstream controllers can call `SecurityContextHolder.getContext().getAuthentication()` to get the user identity.

If JWT validation fails, we call `response.sendError(401)` and return — the request never reaches the controller.

Q: What is `@Transactional` and what happens if you forget it?

A: `@Transactional` is a Spring annotation that wraps a method in a database transaction. Before the method executes, Spring begins a transaction (BEGIN). If the method completes normally, it commits (COMMIT). If an unchecked exception is thrown, it rolls back (ROLLBACK).

If you forget `@Transactional` on a service method that does multiple DB writes:

```
// BAD — no @Transactional
public void upgradeUserToPro(Long userId, String paymentId) {
    userRepository.updatePlanToPro(userId);       // COMMIT immediately
    paymentRepository.recordPayment(paymentId);   // Fails with DB error
    // Result: user is Pro but payment is never recorded — data inconsistency
}

// GOOD — with @Transactional
@Transactional
public void upgradeUserToPro(Long userId, String paymentId) {
    userRepository.updatePlanToPro(userId);
    paymentRepository.recordPayment(paymentId);
    // If second call fails, BOTH are rolled back — atomic.
}
```

We also use `@Transactional(readOnly = true)` for query-only methods. This tells Hibernate not to track entity changes (dirty checking), reducing memory overhead.

Q: Explain `@Modifying` and when you'd use it over `save()`.

A: `@Modifying` marks a `@Query` as a DML statement (UPDATE or DELETE) rather than a SELECT. It's used for bulk operations.

Example — invalidating all sessions for a user:

```
// Option A: save() approach — fetches all entities, modifies, re-saves (N+1 problem)
User user = userRepository.findById(id).orElseThrow();
user.setSessionVersion(user.getSessionVersion() + 1);
userRepository.save(user);
// SQL: SELECT * FROM users WHERE id = ?
//      UPDATE users SET session_version = ? WHERE id = ?
// Two round-trips. Loads entire entity into memory.

// Option B: @Modifying — single atomic SQL
@Modifying
@Transactional
@Query("UPDATE User u SET u.sessionVersion = u.sessionVersion + 1 WHERE u.id = :id")
void incrementSessionVersion(@Param("id") Long id);
```

```
// SQL: UPDATE users SET session_version = session_version + 1 WHERE id = ?
// One round-trip. No entity loaded into memory. Atomic at DB level.
```

`@Modifying` is the correct approach whenever you don't need the updated entity in memory after the operation.

---

5. DATABASE DESIGN

The database is powered by **PostgreSQL 16**, accessed via Hibernate/JPA, and modeled for high-volume analytics tracking and robust consistency.

Schema Philosophy

All tables follow a standardized physical column ordering for DBA readability and enterprise norm alignment:

```
CREATE TABLE tts_history (
    -- 1. Primary Key
    id              BIGINT          NOT NULL PRIMARY KEY,

    -- 2. Foreign Keys (always indexed)
    user_id         BIGINT          NOT NULL REFERENCES users(id),

    -- 3. Core Business Columns
    voice_id        VARCHAR(100)    NOT NULL,
    text_snippet    VARCHAR(500)    NOT NULL,
    output_format   VARCHAR(10)     NOT NULL,
    character_count INTEGER         NOT NULL,
    engine_used     VARCHAR(20)     NOT NULL,

    -- 4. Status Flags
    is_successful   BOOLEAN         NOT NULL DEFAULT TRUE,

    -- 5. Audit Fields
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT NOW(),

    -- 6. Optimistic Locking
    version         BIGINT          NOT NULL DEFAULT 0
);

CREATE INDEX idx_tts_history_user_id ON tts_history(user_id);
```

Core Design Decisions

1. Sequence-Based IDs with Pooled Optimizer (`allocationSize = 50`)

```
@SequenceGenerator(name = "user_seq", sequenceName = "users_seq", allocationSize = 50)
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "user_seq")
private Long id;
```

Why not `GenerationType.IDENTITY`?

With `IDENTITY`, the DB generates the ID on INSERT. This means Hibernate must execute `INSERT` immediately to get the ID back — it can't batch multiple inserts together.

With `SEQUENCE` + `allocationSize = 50`: Hibernate calls `nextval('users_seq')` once and receives a block of 50 IDs (e.g., 1–50). It allocates these from JVM memory, never hitting the DB for IDs again until the block is exhausted. 50 inserts = 1 DB round-trip for IDs instead of 50.

Performance impact: ~98% reduction in ID-generation DB calls during bulk inserts.

Tradeoff: Gaps appear in the ID sequence if the app restarts before exhausting the allocated block. For example, IDs 1, 2, 3 are used, app restarts, next block starts at 51. IDs 4–50 are lost forever. This is acceptable — Primary Keys are not meant to be business-meaningful sequential numbers.

2. Interface Projections for N+1 Prevention

```
// Projection interface — only the columns we need
public interface UserSessionProjection {
    Long getId();
    Integer getSessionVersion();
```

```
    Boolean getHasNaturalVoiceAccess();
}

// Repository method
Optional<UserSessionProjection> findProjectedById(Long id);
```

Generated SQL:

```
-- Without projection
SELECT id, email, password_hash, plan, created_at, updated_at, version,
       session_version, has_natural_voice_access FROM users WHERE id = ?

-- With projection
SELECT id, session_version, has_natural_voice_access FROM users WHERE id = ?
```

For the JWT validation hot-path (every authenticated request), this executes hundreds of times per second. Loading 3 columns vs 9+ columns reduces network transfer and Hibernate object allocation significantly.

3. Soft Deletes vs. Hard Deletes

We use **soft deletes** (`is_active = false`) rather than `DELETE` statements for user records.

Why?

- Audit trail: regulators, billing systems, and fraud investigation require knowing an account existed.
- Referential integrity: `tts_history` has a foreign key to `users`. Hard-deleting a user would require cascading deletes of all their history, or orphaned records.
- Recovery: Users accidentally deleting their accounts can be restored by flipping `is_active = true`.

Tradeoff: Queries must always include `WHERE is_active = TRUE` or use Hibernate's `@Where` annotation to filter automatically. Forgetting this filter returns "deleted" data.

Interview Questions — Database

Q: What is the N+1 query problem and how did you prevent it?

A: The N+1 problem occurs when fetching a list of entities triggers one query for the list plus one additional query per entity to load a related entity.

```
// Example causing N+1
List<User> users = userRepository.findAll(); // Query 1: SELECT * FROM users (returns 100 users)
for (User user : users) {
    System.out.println(user.getHistories().size()); // Query 2..101: SELECT * FROM tts_history WHERE user_id = ?
}
// Total: 101 queries for what should be 2
```

Solutions we use:

1. **`@EntityGraph`** — for cases where we genuinely need the related data, forces a JOIN FETCH in one query.
2. **Interface Projections** — when we only need specific columns and don't need relationships at all.
3. **Lazy loading + explicit JOIN FETCH in JPQL** — for controlled relationship loading.
4. **`getReferenceById`** — for writes where we only need the FK, not the entity data.

Q: Explain PgBouncer and why it's critical for containerized applications.

A: PostgreSQL handles connections by forking a new OS process per client connection. Each process consumes ~5-10MB RAM. A PostgreSQL instance might support 100 concurrent connections before running out of memory.

In a containerized environment:

- Render might spin up 3 instances of our Spring Boot app
- Each has HikariCP with max pool size 10
- That's 30 physical connections to PostgreSQL

But if Render scales to 10 instances under load: 100 connections — hitting PostgreSQL's limit.

PgBouncer (Supabase Session Pooler) acts as a connection multiplexer:

```
App Instance 1 (10 connections)  ┐
App Instance 2 (10 connections)  ┼→ PgBouncer → [3-5 actual PostgreSQL connections]
App Instance 3 (10 connections)  ┘
```

PgBouncer maintains a small pool of real PostgreSQL connections and queues/routes application requests through them. 30 "virtual" app connections map to 5 real DB connections. This allows PostgreSQL to serve far more application nodes than its native connection limit would otherwise allow.

---

6. SECURITY ARCHITECTURE

SpeakIT adopts a "Zero Trust" and "Security-First" approach, treating all input and state as potentially malicious.

Authentication Flow — Stateless JWT with Stateful Invalidation

The Standard JWT Problem: A JWT is self-contained and stateless. If a token is stolen or a user changes their password, the old token remains valid until it expires (typically 1–24 hours). This is unacceptable for a SaaS with billing.

The SpeakIT Solution — Session Versioning:

```
JWT Payload: { userId: 42, sessionVersion: 7, exp: ... }
DB:          users table: { id: 42, session_version: 7 }

On every request:
  Filter reads sessionVersion from JWT (no DB call yet)
  Filter fetches UserSessionProjection from DB (ONE indexed query)
  Filter compares: JWT.sessionVersion == DB.session_version?
    → Match: proceed
    → Mismatch: 401 Unauthorized

On "Logout from all devices":
  UPDATE users SET session_version = session_version + 1 WHERE id = 42
  → All old tokens (with version 7) are instantly invalid
  → No Redis. No token blacklist. One DB update.
```

Why this is better than a token blacklist: A blacklist grows infinitely and must be checked on every request. Session versioning is O(1) — one indexed SELECT.

Deep Hardening

> **Note:** Input sanitization (Jsoup HTML stripping, prompt-injection filtering, abuse pattern detection) and its interaction with rate limiting are covered in full in **Section 7: Rate Limiting & Traffic Governance** under the "Input Sanitization & Abuse Detection" subsection.

MDC (Mapped Diagnostic Context) Tracing

```
// In RequestLoggingFilter
String requestId = UUID.randomUUID().toString().substring(0, 8);
MDC.put("X-Request-ID", requestId);
response.addHeader("X-Request-ID", requestId);
// ... chain.doFilter(request, response)
// finally: MDC.clear() — prevents memory leaks in thread pools
```

Logback pattern in `logback-spring.xml`:

```
<pattern>%d{ISO8601} [%X{X-Request-ID}] [%thread] %-5level %logger{36} - %msg%n</pattern>
```

Result:

```
2025-07-01 14:23:01 [a3f9b2c1] [http-nio-8080-exec-5] INFO  JwtFilter - JWT valid for userId=42
2025-07-01 14:23:01 [a3f9b2c1] [http-nio-8080-exec-5] INFO  PollyService - Requesting NEURAL engine
2025-07-01 14:23:01 [a3f9b2c1] [http-nio-8080-exec-5] INFO  TtsController - Streaming 48,320 bytes
```

All three log lines share the same `X-Request-ID`. In a system handling 1,000 concurrent requests, you can grep `[a3f9b2c1]` and see the exact execution path for one request.

Interview Questions — Security

Q: What is JWT and what are its components?

A: JWT (JSON Web Token) is a compact, URL-safe token format used for stateless authentication. It has three Base64URL-encoded parts separated by dots:

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOjQyLCJzZXNzaW9uVmVyc2lvbiI6N30.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
     HEADER                          PAYLOAD                              SIGNATURE
```

- **Header:** `{ "alg": "HS256", "typ": "JWT" }` — algorithm used to sign
- **Payload (Claims):** `{ "userId": 42, "sessionVersion": 7, "exp": 1720000000 }` — business data
- **Signature:** `HMAC-SHA256(base64(header) + "." + base64(payload), secret)` — tamper detection

Key point: The payload is Base64URL-encoded, NOT encrypted. Anyone can decode it. The signature only proves it wasn't tampered with. **Never put sensitive data in JWT payload.**

In SpeakIT's filter:

1. Split on `.`
2. Verify the signature using our secret key
3. Check `exp` claim against current timestamp
4. Extract `userId` and `sessionVersion` from payload
5. Validate `sessionVersion` against DB

Q: What is CORS and how did you configure it?

A: CORS (Cross-Origin Resource Sharing) is a browser security mechanism. When JavaScript on `mohitur-speakit.vercel.app` (origin A) makes an HTTP request to `text-to-speech-java-backend.onrender.com` (origin B), the browser first sends a "preflight" `OPTIONS` request. The server must respond with specific headers allowing the request, or the browser blocks it.

Our Spring Boot configuration:

```
# application.properties
cors.allowed-origins=${CORS_ALLOWED_ORIGINS:http://localhost:4200}
cors.allowed-methods=GET,POST,PUT,DELETE,OPTIONS
cors.allowed-headers=Authorization,Content-Type,X-Request-ID
cors.exposed-headers=X-Request-ID,Retry-After
cors.max-age=3600
```

`CORS_ALLOWED_ORIGINS` is an environment variable. On Render's prod environment it's set to `https://mohitur-speakit.vercel.app`. Locally it defaults to `http://localhost:4200`. This means a malicious third-party website cannot make authenticated calls to our API using a victim's browser session.

---

7. RATE LIMITING & TRAFFIC GOVERNANCE

This section is the single authoritative source for all rate limiting concerns in SpeakIT — covering the threat model, AOP implementation, algorithm design, identity model, input sanitization, abuse detection, graceful degradation, and the Redis migration path.

7.1 Why Rate Limiting Is Both a Security AND Financial Concern

SpeakIT is not a simple CRUD app. Every synthesis request invokes AWS Polly, which costs real money per character. This creates two failure modes that rate limiting must address simultaneously:

```
Security failure:  Attacker floods /login → brute-forces accounts
Financial failure: Attacker floods /synthesize → drains AWS Polly budget → startup runway gone
```

Rate limiting in SpeakIT is therefore **dual-purpose infrastructure** — it protects both the system's integrity and the company's bank account.

7.2 Why IP-Only Rate Limiting Was Rejected

The naive approach most tutorials demonstrate is:

```
IP Address → Request Counter → Block when counter > N
```

This fails in every real-world attack scenario:

```
Attacker bypass techniques:
  - Residential proxy pools (Luminati, Oxylabs) → new IP per request, all legitimate-looking
  - Mobile carrier NATs → thousands of users share one IP (blocking it = mass false positives)
  - Cloud VM rotation → AWS/GCP spin up new IPs in seconds
  - Tor exit nodes → IP changes per circuit
  - VPN chaining → Layer multiple VPN hops

False positive example:
  A university's 10,000 students share one egress IP.
  One student abuses the API → entire university is blocked.
  IP-only limiting is broken by design.
```

7.3 The Composite Identity Model

SpeakIT builds a **composite identity profile** per request, not a single IP lookup.

REQUEST IDENTITY SIGNALS	
Signal	Source
CF-Connecting-IP	Cloudflare header (trusted edge authority)
X-Forwarded-For	Fallback proxy chain
Servlet remote addr	Final fallback (direct TCP)
User-Agent	Browser/client string
JWT → User ID	Authenticated identity (strongest signal)
Endpoint	Which protection zone applies
Auth state	Anonymous vs authenticated

```
Resolution logic in RateLimitAspect:
  if (authenticated):
    key = "user:" + userId          ← JWT-bound. Proxy rotation irrelevant.
  else:
    key = "anon:" + hash(CF-IP + User-Agent)   ← Lightweight fingerprint
```

Why trust `CF-Connecting-IP` over `X-Forwarded-For`?

`X-Forwarded-For` is trivially spoofed if you POST directly to the origin server. An attacker can send: `X-Forwarded-For: 1.2.3.4` and bypass IP-based checks. Because all traffic flows through Cloudflare first, `CF-Connecting-IP` is set by Cloudflare's infrastructure — not by the client. The origin server trusts Cloudflare as the ingress authority and uses `CF-Connecting-IP` as the ground truth.

7.4 Defense-in-Depth Architecture

Rate limiting is one layer in a multi-layer protection stack. Each layer addresses a different attack category:

```
Internet
   ↓
[Layer 1] Cloudflare Edge / WAF
          → DDoS volumetric attack absorption (L3/L4)
          → Bot score filtering (JS challenge for suspicious traffic)
          → Geo-blocking (if enabled)
          → SSL termination (TLS 1.2+ enforced)
   ↓
[Layer 2] Trusted Proxy Header Resolution (RateLimitAspect)
          → CF-Connecting-IP > X-Forwarded-For > Servlet remote addr
          → Prevents X-Forwarded-For spoofing at origin
   ↓
[Layer 3] Spring Security Filter Chain
          → JWT signature verification
          → Session version check (stateful invalidation)
          → 401 short-circuit if auth fails (request never hits controller)
   ↓
[Layer 4] RateLimitAspect — @RateLimited (Bucket4j)
          → Token bucket enforcement per identity+zone
          → 429 + Retry-After if bucket exhausted
   ↓
[Layer 5] Input Sanitization & Abuse Detection (TtsController)
          → Jsoup HTML stripping
          → Prompt injection / abuse pattern regex scan
          → Character limit validation (plan-aware)
   ↓
[Layer 6] AWS Polly Invocation
```

```
          → Only reached if all 5 layers above pass
          → Financial kill switch via AWS Budgets + Lambda as final backstop
```

Each layer has a specific responsibility. An attacker who bypasses Layer 1 (unlikely at Cloudflare scale) still hits Layers 2–6. This is classical defense-in-depth.

7.5 AOP Implementation — @RateLimited Annotation

Rate limiting is wired into the application via **Aspect-Oriented Programming (AOP)**, not inline controller code.

Why AOP?

Without AOP, every protected endpoint would mix infrastructure concerns with business logic:

```
// BAD — violates Single Responsibility Principle
@PostMapping("/synthesize")
public ResponseEntity<?> synthesize(@RequestBody TtsRequest req, HttpServletRequest request) {
    // Infrastructure concern polluting business method:
    String userId = (String) request.getAttribute("userId");
    Bucket bucket = buckets.computeIfAbsent(userId, k -> buildBucket());
    ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
    if (!probe.isConsumed()) {
        long retry = probe.getNanosToWaitForRefill() / 1_000_000_000;
        response.addHeader("Retry-After", String.valueOf(retry));
        return ResponseEntity.status(429).build();
    }
    // Actual business logic finally starts here...
}
```

With AOP:

```
// GOOD — controller is clean, rate limiting is externalized
@PostMapping("/synthesize")
@RateLimited(action = RateLimitAction.TTS_SYNTHESIS)
public ResponseEntity<?> synthesize(@RequestBody TtsRequest req) {
    // Pure business logic. Rate limiting happens transparently before this runs.
    return pollyService.synthesize(req);
}
```

How the Aspect works:

```
@Aspect
@Component
public class RateLimitAspect {

    private final ConcurrentHashMap<String, Bucket> buckets = new ConcurrentHashMap<>();

    @Around("@annotation(rateLimited)")
    public Object enforceRateLimit(ProceedingJoinPoint joinPoint, RateLimited rateLimited)
            throws Throwable {

        // 1. Build composite identity key
        String key = resolveIdentityKey(rateLimited.action());

        // 2. Get or create bucket for this identity (thread-safe via ConcurrentHashMap)
        Bucket bucket = buckets.computeIfAbsent(key, k -> buildBucket(rateLimited.action()));

        // 3. Attempt to consume one token
        ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);

        if (probe.isConsumed()) {
            return joinPoint.proceed();   // Token consumed — proceed to controller method
        }

        // 4. Bucket empty — compute exact retry time from nanosecond refill ETA
        long retryAfterSeconds = probe.getNanosToWaitForRefill() / 1_000_000_000;
        HttpServletResponse response = getCurrentHttpResponse();
        response.addHeader("Retry-After", String.valueOf(retryAfterSeconds));
        throw new TooManyRequestsException(retryAfterSeconds);
    }
}
```

Open-Closed Principle in action: Adding rate limiting to a new endpoint requires exactly one annotation. The `RateLimitAspect` class never needs modification. This is the textbook definition of "open for extension, closed for modification."

7.6 RateLimitAction — Differentiated Protection Zones

A single global rate limit is a design mistake. Different endpoints have fundamentally different threat models and cost profiles. `RateLimitAction` is an enum that maps each endpoint category to its own bucket configuration:

```
public enum RateLimitAction {
    AUTH_FLOW,       // login, register, forgot-password
    TTS_SYNTHESIS,   // POST /api/tts/synthesize — most expensive
    PUBLIC_API       // GET /api/tts/voices, contact form, ping
}
```

Zone	Endpoints	Identity Key	Bucket Configuration
AUTH_FLOW	/login /register /forgot-password	hash(IP + UserAgent) (no JWT yet)	5 tokens, refill 5/min Extremely restrictive. Stops brute force at ~5 tries.
TTS_SYNTHESIS	/synthesize /synthesize-stream	"user:" + userId (JWT-bound)	30 burst, refill 10/min Allows natural bursting. Caps sustained automated abuse.
PUBLIC_API	/voices /ping /contact	hash(IP + UserAgent)	60 tokens, refill 60/min Lightweight. Allows crawlers but stops scrapers.

Why AUTH_FLOW is fingerprint-based, not user-based: At the login endpoint, we don't have a user identity yet — the attacker is trying to *discover* one. We can only fingerprint by IP + User-Agent. This is intentionally restrictive: 5 failed login attempts per minute is more than enough for a legitimate user who forgot their password, and far too few for a credential-stuffing bot.

Why TTS_SYNTHESIS is account-bound: Every synthesis request costs money. An attacker who creates 10 accounts to multiply their quota is still bounded — 10 accounts × 30 burst = 300 Polly calls before any sustained throttling kicks in. That's an acceptable ceiling given the account creation friction.

7.7 Token Bucket Algorithm — Why It's the Right Choice

Three rate limiting algorithms exist. Here's why Token Bucket was chosen:

Algorithm 1: Fixed Window Counter (rejected)

```
Window: 0–60s   → counter = 0 ... 10 → BLOCK at 10
Window: 60–120s → counter resets to 0

Problem — "Reset cliff" abuse:
  Attacker sends 10 requests at t=59s → allowed (window 1)
  Attacker sends 10 requests at t=61s → allowed (window 2 reset)
  Result: 20 requests in 2 seconds. The "limit" is defeated.
```

Algorithm 2: Sliding Window Log (rejected)

```
Track timestamp of every request in a log.
On new request: count requests in last 60s from log.

Accurate, but: stores O(N) timestamps per user. At 10,000 users × 30 requests = 300,000 log entries.
Memory footprint unacceptable.
```

Algorithm 3: Token Bucket (chosen — Bucket4j)

```
Each user has a bucket with capacity C and refill rate R.
Bucket starts full.
Each request: consume 1 token. If 0 tokens → reject.
Tokens refill continuously at rate R (not in windows).

SpeakIT TTS config: C = 30, R = 10/minute

Scenario A — legitimate user (rapid testing):
  t=0:  30 requests burst → all succeed (30 → 0 tokens)
  t=1m: 10 more requests → all succeed (refill → 10, consume 10)
  t=2m: 10 more requests → all succeed

Scenario B — attacker (sustained flooding):
  t=0:  30 requests → succeed (burst consumed)
  t=1s: 100 more requests → 100 BLOCKED (bucket empty, refill not yet)
```

```
  t=1m: 10 requests → succeed (refill)
  Sustained rate locked at 10/minute forever.

Scenario C — edge case (user walks away, comes back):
  t=0:  5 requests (25 tokens remain)
  t=10m: bucket is now full again (10 tokens × 10 min = capped at 30)
  t=10m: 30 burst available again → good UX for returning user
```

Token bucket accommodates **human burst patterns** while capping **sustained automated throughput**. This is the correct algorithm for a developer-facing API where "try 5 voices quickly" is a valid use case.

7.8 Bucket4j — Implementation Details

Why Bucket4j over hand-rolling or other libraries?

- **Nanosecond precision:** `probe.getNanosToWaitForRefill()` gives exact refill timing → enables precise `Retry-After` headers.
- **Lock-free CAS concurrency:** No `synchronized` blocks. Thread-safe via CPU-level atomic operations.
- **Multiple backend support:** In-memory `ConcurrentHashMap` now → Redis/JCache later. Same API, different backend.
- **Proven library:** Used in production at large-scale Java SaaS companies. Not a toy implementation.

Bucket construction:

```
private Bucket buildBucket(RateLimitAction action) {
    return switch (action) {
        case AUTH_FLOW -> Bucket.builder()
            .addLimit(Bandwidth.builder()
                .capacity(5)
                .refillGreedy(5, Duration.ofMinutes(1))
                .build())
            .build();

        case TTS_SYNTHESIS -> Bucket.builder()
            .addLimit(Bandwidth.builder()
                .capacity(30)                                   // burst capacity
                .refillGreedy(10, Duration.ofMinutes(1))        // sustained refill
                .build())
            .build();

        case PUBLIC_API -> Bucket.builder()
            .addLimit(Bandwidth.builder()
                .capacity(60)
                .refillGreedy(60, Duration.ofMinutes(1))
                .build())
            .build();
    };
}
```

`refillGreedy` vs `refillIntervally`:

- `refillGreedy(10, 1min)` — adds tokens continuously: 1 token every 6 seconds. Smoother traffic shaping.
- `refillIntervally(10, 1min)` — adds all 10 tokens at once every 60 seconds. Creates mini reset-cliff within the bucket.

We use `refillGreedy` for smoother sustained throttling.

7.9 Thread Safety — CAS (Compare-And-Swap)

In a multi-threaded Spring Boot server, many threads hit the same user's rate limit bucket simultaneously. Bucket4j handles this via lock-free **Compare-And-Swap (CAS)**:

```
Two threads, both trying to consume 1 token from bucket with 5 tokens:

Thread 1: read state = 5 tokens
Thread 2: read state = 5 tokens  (concurrent read — both see 5)

Thread 1: CAS(expected=5, new=4) → CPU checks: current==5? YES → sets to 4 → SUCCESS
Thread 2: CAS(expected=5, new=4) → CPU checks: current==5? NO (it's 4) → FAIL → retry
Thread 2: re-reads state = 4
Thread 2: CAS(expected=4, new=3) → CPU checks: current==4? YES → sets to 3 → SUCCESS

Final state: 3 tokens. Both threads consumed correctly. Zero locks. Zero blocking.
```

CAS is a single atomic CPU instruction (`LOCK CMPXCHG` on x86). It's orders of magnitude faster than `synchronized` blocks which require OS-level mutex acquisition. This is why Bucket4j performs at millions of operations per second without thread contention.

7.10 Input Sanitization & Abuse Detection

Even though AWS Polly is not an LLM, public AI endpoints attract automated scanners that probe for prompt injection, test AI infrastructure, and look for exploitable attack surfaces. These scanners don't know or care that Polly just converts text to speech.

Sanitization Before Validation (Order Matters)

```
// TtsController — the order of these two lines is a deliberate security decision
String sanitizedText = Jsoup.clean(rawText, Safelist.none());  // Step 1: Strip all HTML
if (sanitizedText.length() > planCharacterLimit) {              // Step 2: Check length
    throw new CharacterLimitExceededException();
}
```

Why sanitize FIRST, validate SECOND?

Consider this attack:

```
Attacker's input: "Hello<script>alert(1)</script><div style='display:none'>AAAA...AAAA</div>"

Without sanitize-first:
  Raw length = 12 (spoken) + 5000 (hidden HTML padding) = 5012 chars
  5012 > 3000 (Pro limit) → "Character limit exceeded" error
  Attacker learns: they can probe limits with hidden characters

  OR:

  Raw length = 15 (actual text) + hidden HTML = looks like 200 chars (Free limit OK)
  String stored/reflected → XSS payload executes in admin dashboard

With sanitize-first:
  Jsoup strips ALL HTML → clean string = "Hello" (5 chars)
  5 < 200 → passes Free limit check
  Polly speaks "Hello" — attacker accomplished nothing
  XSS vector eliminated entirely
```

Abuse Pattern Detection

Before invoking Polly, the text is scanned against a regex pattern list:

```
private static final List<Pattern> ABUSE_PATTERNS = List.of(
    Pattern.compile("(?i)ignore (previous|all) instructions"),
    Pattern.compile("(?i)act as (a |an )?(system|admin|root)"),
    Pattern.compile("(?i)you are now"),
    Pattern.compile("(?i)jailbreak"),
    Pattern.compile("(?i)<\\?.*\\?>"),          // PHP injection probes
    Pattern.compile("(?i)\\$\\{.*\\}")           // Template injection probes
);

public boolean isAbusive(String text) {
    return ABUSE_PATTERNS.stream().anyMatch(p -> p.matcher(text).find());
}
```

If any pattern matches, the request is rejected **before Polly is invoked** — zero AWS cost incurred, abuse attempt logged with the MDC request ID for forensics.

Why does this matter even though Polly isn't an LLM?

1. Automated AI scanners (red-team bots, bug bounty hunters, security researchers) sweep all AI-adjacent endpoints with prompt injection payloads. Rejecting them early saves Polly TPS quota and reduces noise in logs.
2. If we ever add an LLM layer (voice narration with GPT-4 for content generation), the sanitization layer is already in place.
3. Early rejection = zero cost. Late rejection = we paid Polly and then threw away the result.

7.11 Retry-After Header — Enterprise Graceful Degradation

The industry-standard bad practice:

```
HTTP/1.1 429 Too Many Requests

{"error": "Too many requests"}
```

Client behavior: implement exponential backoff with random jitter. Different clients implement this differently. Result: **retry storms** — clients all retry at slightly different intervals, creating amplified load spikes.

SpeakIT's approach:

```
// RateLimitAspect — nanosecond precision from Bucket4j
ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
if (!probe.isConsumed()) {
    long nanosToWait = probe.getNanosToWaitForRefill();
    long retryAfterSeconds = TimeUnit.NANOSECONDS.toSeconds(nanosToWait) + 1; // +1 for safety margin

    HttpServletResponse response = ((ServletRequestAttributes)
        RequestContextHolder.getRequestAttributes()).getResponse();
    response.addHeader("Retry-After", String.valueOf(retryAfterSeconds));
    response.addHeader("X-RateLimit-Limit", String.valueOf(bucketCapacity));
    response.addHeader("X-RateLimit-Remaining", "0");
    response.addHeader("X-RateLimit-Reset", String.valueOf(
        Instant.now().plusSeconds(retryAfterSeconds).getEpochSecond()
    ));

    throw new TooManyRequestsException(retryAfterSeconds);
}
```

```
HTTP/1.1 429 Too Many Requests
Retry-After: 42
X-RateLimit-Limit: 30
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1720000042
X-Request-ID: a3f9b2c1

{
  "error": "Rate limit exceeded",
  "retryAfterSeconds": 42,
  "message": "You've used your synthesis quota. Try again in 42 seconds."
}
```

Angular frontend reads `Retry-After` and renders:

```
□ Rate limit reached. Retrying in 42s...  [███████░░░░░░░]
```

A live countdown timer. The user knows exactly when they can try again — no ambiguity, no frustration, no support tickets.

Why this is better: Clients that respect `Retry-After` will retry exactly when capacity is available. No retry storms. No amplification. Load resumes smoothly as buckets refill.

7.12 ConcurrentHashMap Bucket Store & The Redis Migration Path

Current implementation:

```
private final ConcurrentHashMap<String, Bucket> buckets = new ConcurrentHashMap<>();
```

Why `ConcurrentHashMap` and not `HashMap`?

`HashMap` is not thread-safe. Concurrent writes can corrupt the internal hash table structure, causing infinite loops or data loss. `ConcurrentHashMap` uses segment-level locking — reads are fully concurrent, writes lock only the affected segment (not the whole map). This provides high throughput without synchronization overhead on the happy path.

The known limitation — single-node scope:

```
With 2 Spring Boot instances:

  User makes 35 requests:
    20 → Node A: A's bucket: 30 → 10 tokens remaining
    15 → Node B: B's bucket: 30 → 15 tokens remaining

  User consumed 35 requests but should have been throttled at 30.
  Effective quota is now (30 × num_nodes).
```

The documented Redis migration (Bucket4j-Redis):

```
// Step 1: Add dependency
// io.github.bucket4j:bucket4j-redis

// Step 2: Configure Redisson client
@Bean
public RedissonClient redissonClient() {
    Config config = new Config();
    config.useSingleServer().setAddress("redis://" + redisHost + ":6379");
    return Redisson.create(config);
}

// Step 3: Replace ConcurrentHashMap with Redis ProxyManager
@Bean
public ProxyManager<String> proxyManager(RedissonClient redissonClient) {
    return Bucket4jRedis.casBasedBuilder(redissonClient).build();
}

// Step 4: Bucket creation unchanged — same API, shared backend
Bucket bucket = proxyManager.builder()
    .addLimit(buildBandwidth(action))
    .build(identityKey);
// Now all nodes share the same bucket counter in Redis.
```

Why we deferred Redis:

Factor	Decision
Single-node deployment	In-memory is faster and simpler
Cost	No Redis instance to provision or pay for
Operational complexity	No Redis availability to manage, no connection pool config
Startup velocity	Ship product first, add distributed infra when actually needed
Migration effort	When needed: add dependency + 20 lines of config. The Aspect code doesn't change.

This is a deliberate, documented tradeoff — not a gap. The migration is 30 minutes of work when we hit it.

Interview Questions — Rate Limiting

Q: Walk me through exactly what happens when a user hits the rate limit on /synthesize.

A:

1. `POST /synthesize` arrives. Passes through `RequestLoggingFilter` (X-Request-ID assigned) and `JwtAuthenticationFilter` (JWT validated, userId=42 injected).
2. Spring AOP intercepts the controller method because it's annotated `@RateLimited(action = TTS_SYNTHESIS)`.
3. `RateLimitAspect.enforceRateLimit()` resolves identity key: `"user:42"`.
4. `buckets.computeIfAbsent("user:42", ...)` returns the existing Bucket (or creates one if first request).
5. `bucket.tryConsumeAndReturnRemaining(1)` is called. Returns `ConsumptionProbe`.
6. `probe.isConsumed()` → `false` (bucket empty).
7. `probe.getNanosToWaitForRefill()` → e.g., `42,000,000,000` nanoseconds → 42 seconds.
8. `Retry-After: 42` header added to response. `TooManyRequestsException(42)` thrown.
9. `GlobalExceptionHandler` catches it → serializes `{ "error": "Rate limit exceeded", "retryAfterSeconds": 42 }`.
10. HTTP 429 returned to Angular. Angular reads `Retry-After`, starts countdown timer.
11. At t+42s, Angular auto-retries. Bucket has refilled (~7 tokens at 10/min rate). Request proceeds.

Q: Your rate limiting uses an in-memory ConcurrentHashMap. What happens when you add a second backend instance?

A: Each node has its own independent bucket store. With 2 nodes:

```
User makes 35 requests:
  20 → Node A (bucket: 30 → 10 remaining)
  15 → Node B (bucket: 30 → 15 remaining)

User was never rejected — 35 > 30. Limit is defeated by horizontal scale.
```

This is a documented, accepted limitation for the current single-node deployment. The fix is Bucket4j + Redis (see Section 7.12). We defer it because the migration is minimal and the current deployment has no horizontal scaling.

Q: How does Bucket4j guarantee thread safety without locks?

A: Bucket4j uses **Compare-And-Swap (CAS)** — a single atomic CPU instruction (`LOCK CMPXCHG` on x86). No `synchronized`, no `ReentrantLock`, no OS mutex.

```
Thread 1 reads: tokens = 5
Thread 2 reads: tokens = 5  ← concurrent read, both see 5

Thread 1: CAS(expected=5, new=4) → current IS 5 → atomically set to 4 → SUCCESS
Thread 2: CAS(expected=5, new=4) → current IS 4 (not 5) → FAIL → retry
Thread 2: re-reads = 4
Thread 2: CAS(expected=4, new=3) → current IS 4 → set to 3 → SUCCESS

Final: 3 tokens. Both consumed exactly 1 each. No data race. No blocking.
```

CAS retry loops are fast because contention on a single value is rare and short-lived. This is why Bucket4j throughput is measured in millions of ops/second.

Q: What algorithm does Bucket4j use and why did you choose it over fixed-window?

A: Token Bucket. The key advantage over fixed windows is that it eliminates the "reset cliff" — with fixed windows, an attacker sends 10 requests at second 59 and 10 more at second 61, effectively getting 20 in 2 seconds. Token bucket refills continuously, so there's no hard boundary to game.

Additionally, token bucket naturally models **human burst behavior**: a developer rapidly testing 5 different voices in 30 seconds is legitimate usage. A fixed window would throttle this. The burst capacity (30 tokens) accommodates the human; the sustained refill rate (10/minute) punishes the bot.

Q: Why is AUTH_FLOW rate limited by fingerprint but TTS_SYNTHESIS by user ID?

A: At the `/login` endpoint, we have no user identity yet — that's what the attacker is trying to obtain. We can only fingerprint on what's observable: IP + User-Agent. This is intentionally crude but effective at stopping automated credential stuffing bots that reuse the same browser fingerprint.

At `/synthesize`, we have a verified JWT with a stable user ID. Using the user ID as the rate limit key means: VPN rotation, Incognito mode, new device, new browser — none of it helps the attacker. They are the same "user:42" regardless of what network they come from. The quota follows the account identity, not the network identity.

Q: You mentioned prompt injection filtering even though Polly isn't an LLM. Why?

A: Three reasons:

1. **Automated scanners don't discriminate.** Red-team bots, bug bounty automation, and AI security scanners probe every AI-adjacent endpoint with injection payloads. They send "Ignore previous instructions" to AWS Polly endpoints the same way they'd send it to GPT-4. Filtering these saves Polly quota and keeps logs clean.

2. **Cost conservation.** A rejected request costs zero. A request that reaches Polly costs money. If we can reject 500 bot probes per day before they hit Polly, that's measurable cost savings.

3. **Future-proofing.** If we add an LLM layer for content generation, the sanitization pipeline is already in place. No retrofitting required.

---

8. DEVOPS & DEPLOYMENT

Infrastructure Overview

```
Frontend: Vercel (Global Edge Network, static SPA, auto HTTPS)
Backend:  Render (Docker container, auto-deploy from GitHub main)
Database: Supabase (PostgreSQL 16 + PgBouncer Session Pooler)
DNS/CDN:  Cloudflare (DNS, DDoS, SSL termination, path routing)
Registry: Docker Hub (mohitur/speakit:backend, mohitur/speakit:frontend)
Budget:   AWS Budgets + SNS + Lambda kill switch (PollyBudgetKillSwitch)
CI:       GitHub Actions (keep-alive ping every 25 minutes)
```

Multi-Stage Docker Build

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline   # Cache deps separately for faster rebuilds
COPY src ./src
RUN mvn package -DskipTests

# Stage 2: Runtime (minimal image)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/speakit-*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Why multi-stage? Single-stage builds include the Maven toolchain (~500MB) in the final image. Multi-stage builds copy only the compiled JAR into a minimal JRE image. Result: image size drops from ~600MB to ~150MB. Smaller images = faster Render deployments and less Docker Hub storage.

Why `eclipse-temurin` instead of `openjdk`? `openjdk:17-jdk-slim` is deprecated and no longer receives security patches. Eclipse Temurin (maintained by the Adoptium project, part of Eclipse Foundation) is the community-endorsed, actively patched OpenJDK distribution.

Keep-Alive Architecture

Render's free/hobby instances spin down after 15 minutes of inactivity. A "cold start" takes 30-60 seconds, creating terrible UX for the first user after inactivity.

```
# .github/workflows/keep-alive.yml
name: Keep Backend Alive
on:
  schedule:
    - cron: "*/25 * * * *" # Every 25 minutes
jobs:
  ping:
    runs-on: ubuntu-latest
    steps:
      - run: curl -f ${{ secrets.BACKEND_URL }}/api/auth/ping
```

`/api/auth/ping` is a `@PermitAll` endpoint that returns `{ "status": "ok" }` — it requires no auth and no DB call. The GitHub Action runner is free for public repos. Zero cost keep-alive.

AWS Budget Kill Switch Architecture

```
AWS Budgets (threshold: $X)
  → SNS Topic: PollyBudgetAlert
    → Lambda: PollyBudgetKillSwitch
      → Attaches IAM Deny Policy: PollyEmergencyDeny
        → Effect: Deny all polly:* actions for the app IAM user
          → All Polly calls return AccessDeniedException
          → Spring Boot's GlobalExceptionHandler catches this
          → Returns 503 with user-friendly message
```

Why this matters: An attacker triggering maximum TTS requests (3,000 chars × 10 requests/min) could drain our AWS Polly budget rapidly. This kill switch is a financial circuit breaker — it cuts Polly access before the bill becomes catastrophic. The application degrades gracefully (503) rather than silently racking up charges.

Interview Questions — DevOps

Q: Walk me through what happens when you push to the `main` branch.

A:

1. GitHub receives the push. If branch protection rules are configured, the PR was already reviewed and CI passed.
2. **Vercel (Frontend):** Vercel's GitHub integration detects the push. It runs `npm run build` (which triggers the `prebuild` hook → `set-env.js` → generates `runtime-env.js`). Vercel deploys the compiled `dist/` to its CDN. Takes ~2 minutes. Zero downtime — old version serves until new version is fully deployed, then traffic switches atomically.
3. **Render (Backend):** Render's GitHub integration detects the push. It pulls the new code, runs the Docker build (multi-stage), pushes to internal registry, and starts a new container. During the health check period, the old container keeps serving. Once the new container passes health checks (`/api/auth/ping` returns 200), Render terminates the old container. Takes ~5 minutes.

4. **Zero user impact:** Both deployments are blue-green — old version stays up until new version is healthy.

Q: How do you manage secrets and environment variables across local dev, Docker Compose, and Render?

A: Three environments, three mechanisms — but the application code reads the same environment variable names everywhere:

```
Local Dev:
  .env file (gitignored) → loaded by Docker Compose or IDE
  Example: AWS_ACCESS_KEY_ID=AKIA...

Docker Compose:
  env_file: .env (reads the same .env)
  docker-compose.yml is committed (no secrets)
  .env is gitignored

Render Production:
  Environment Variables set in Render dashboard
  Render injects them at container startup
  Application reads: System.getenv("AWS_ACCESS_KEY_ID")
  Spring reads:      ${AWS_ACCESS_KEY_ID}
```

This is the **12-Factor App** methodology (Factor III: Config). The application binary is identical everywhere; only the environment differs.

---

9. PERFORMANCE OPTIMIZATION

Backend Optimization

In-Memory API Response Caching

The AWS Polly `DescribeVoices` API (which returns the list of available voices) takes 300-800ms to respond. Voice data changes at most once per year (when AWS adds new voices).

```
@Service
public class PollyService {
    private List<Voice> cachedVoices;
    private Instant cacheExpiry;
    private static final Duration CACHE_TTL = Duration.ofHours(24);

    public List<Voice> getAvailableVoices() {
        if (cachedVoices == null || Instant.now().isAfter(cacheExpiry)) {
            cachedVoices = pollyClient.describeVoices(...).voices();
            cacheExpiry = Instant.now().plus(CACHE_TTL);
        }
        return cachedVoices;
    }
}
```

Result: `/api/tts/voices` latency drops from 300-800ms to <1ms for cached responses. 99.9% of requests hit the cache.

Why not Spring Cache (`@Cacheable`)? For a single cache with a fixed TTL, the manual approach is simpler — no `CacheManager` configuration, no cache name strings, no eviction policy setup. When we have 10+ cacheable methods, `@Cacheable` with Caffeine becomes the correct tool.

Streaming Audio Responses

```
// BAD — buffers entire audio in JVM heap
byte[] audio = pollyClient.synthesizeSpeech(...).audioStream().readAllBytes();
response.getOutputStream().write(audio); // Risk: OOM for large files

// GOOD — streams directly, never fully in memory
try (InputStream pollyStream = pollyClient.synthesizeSpeech(...).audioStream()) {
    response.setContentType("audio/mpeg");
    pollyStream.transferTo(response.getOutputStream());
}
```

The `transferTo()` call pipes 8KB chunks at a time from Polly to the HTTP response. The JVM never holds more than one chunk in memory at a time. This allows concurrent generation of many audio files without OOM errors.

Frontend Optimization

Route-Level Lazy Loading

```
// app.routes.ts
export const routes: Routes = [
  {
    path: "tts",
    loadComponent: () =>
      import("./features/tts/tts-studio.component").then(
        (m) => m.TtsStudioComponent,
      ),
    // ↑ TtsStudioComponent's JS is NOT in the initial bundle.
    //   It's downloaded only when the user navigates to /tts
  },
  {
    path: "auth",
    loadComponent: () =>
      import("./features/auth/login.component").then((m) => m.LoginComponent),
  },
];
```

Bundle impact:

- Without lazy loading: Initial download = marketing + auth + TTS studio + all dependencies (~500KB+)
- With lazy loading: Initial download = marketing page only (~80KB). TTS studio downloads on first navigation.

A marketing visitor who never logs in never downloads the TTS Studio code. This significantly improves Core Web Vitals (Largest Contentful Paint, Time to Interactive).

Interview Questions — Performance

Q: What is HikariCP and how did you tune it?

A: HikariCP is Spring Boot's default JDBC connection pool. Instead of creating a new TCP connection to PostgreSQL on every request (expensive: ~100ms handshake), HikariCP maintains a pool of pre-established connections that queries reuse.

Our configuration:

```
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.idle-timeout=600000
spring.datasource.hikari.max-lifetime=1800000
```

Why max-pool-size=10? Supabase's free tier PgBouncer has a limit on concurrent connections. We set 10 to stay safely below that limit. In production on a paid tier, this would scale up.

Why min-idle=2? Keeping 2 connections always warm means requests don't wait for a connection to be established. The pool never drops below 2, so there's always an immediately available connection for traffic spikes.

Q: What is Time to First Byte (TTFB) vs First Contentful Paint (FCP) and how does your architecture affect them?

A:

- **TTFB:** Time from browser sending HTTP GET to receiving the first byte of the response. Affected by: server processing time, network latency, CDN caching.
- **FCP:** Time from navigation start to when the first piece of DOM content renders. Affected by: TTFB + HTML parsing + critical CSS/JS loading.

SpeakIT's architecture optimizations:

- Angular SPA served from Vercel CDN → TTFB <30ms globally (vs 200ms+ from a single origin)
- Precompressed Brotli assets on Vercel → smaller transfer, faster FCP
- Lazy-loaded routes → initial JS bundle is minimal → faster JS parse/execute → faster FCP
- Tailwind CSS tree-shaking → minimal CSS payload (often <10KB) → faster style calculation

---

10. SOFTWARE ENGINEERING CONCEPTS APPLIED

SOLID Principles

Principle	Application in SpeakIT
Single Responsibility	Controllers handle HTTP; Services handle logic; Repos handle data. Each class has one reason to change.
Open-Closed	`@RateLimited` aspect — protect new endpoints by annotation addition, not code modification (see Section 7.5).
Liskov Substitution	Repository interfaces can swap implementations (JPA → JDBC) without breaking service layer.
Interface Segregation	`UserSessionProjection` exposes only 3 fields; controllers never see the full `User` entity's 12 fields.
Dependency Inversion	Services depend on repository interfaces (`UserRepository`), not concrete Hibernate classes. Spring DI wires the implementations.

Design Patterns

Pattern	Where Used	Why
Filter Chain	`JwtAuthenticationFilter`, `RequestLoggingFilter`	Composable, ordered cross-cutting concerns
Proxy	`getReferenceById` returns Hibernate Proxy	Avoids unnecessary SELECT for FK relationships
Strategy	`RateLimitAction` enum — different bucket configs per endpoint	Swappable algorithms for different rate limit zones
Builder	Bucket4j `Bandwidth.builder()`, AWS SDK request builders	Fluent, immutable object construction
Observer / Event	BroadcastChannel for cross-tab logout	Decoupled event propagation without shared state
Decorator	HTTP Interceptors in Angular	Augment HttpClient behavior without modifying it
Aspect (AOP)	`@RateLimited` annotation (see Section 7.5 for full implementation)	Non-functional concerns separated from business logic

---

11. DESIGN DECISION ANALYSIS (Tradeoffs)

Why Spring Boot + Java 21 instead of Node.js?

Dimension	Spring Boot / Java 21	Node.js / Express
Concurrency model	True multi-threading (virtual threads via Project Loom in Java 21)	Single-threaded event loop
Audio streaming	Blocking I/O with virtual threads — handles many concurrent streams efficiently	Non-blocking I/O — also efficient
Type safety	Strong static typing — compiler catches errors	TypeScript helps but runtime surprises possible
Ecosystem	Spring Security, Hibernate, AWS SDK v2 — mature, production-battle-tested	Rapidly evolving, more fragmented
Memory footprint	Higher baseline (~250MB)	Lower baseline (~50MB)
Cold start	Slower (~5s)	Faster (~1s)
Verdict	Better for enterprise SaaS at scale	Better for quick prototypes and serverless

Java 21's **Virtual Threads** (Project Loom) fundamentally change the concurrency equation — you can now have millions of concurrent blocking operations without OS thread overhead. This makes Spring Boot + Java 21 competitive with Node.js for I/O-heavy workloads like audio streaming.

Why Modular Monolith instead of Microservices?

Conway's Law: organizations build systems that mirror their communication structure. A 1-person project building microservices creates distributed system complexity (service discovery, inter-service authentication, distributed tracing, network partitions) with zero team communication benefit.

Microservices are the correct architecture when:

- Different services need to scale independently (e.g., synthesis gets 100x traffic; auth stays flat)
- Different teams own different bounded contexts (Auth team, TTS team)
- Services need different tech stacks (GPU-heavy inference service in Python; user management in Go)

None of these apply currently. The modular monolith gives us:

- Clean domain separation (package-level: `auth`, `tts`, `user`)
- Single deployment pipeline
- Zero network latency between "services"
- Easy refactoring without network contract changes

Migration path: when we hire team 2 (TTS team), extract `PollyService` into a microservice with a REST contract. The domain boundary is already clean.

Why PostgreSQL over MongoDB?

The data model is **highly relational**:

```
User (1) ──────────── (N) TtsHistory
User (1) ──────────── (1) Subscription
Subscription (N) ──── (1) Plan
```

ACID compliance is **non-negotiable** for:

- Billing: charging a user and recording the payment must be atomic
- Rate limiting: session version increments must be atomic and strongly consistent
- Plan management: upgrading a user to Pro must reflect immediately across all nodes

MongoDB's eventual consistency model and lack of multi-document ACID (prior to version 4.0+) make it the wrong choice for financial data. PostgreSQL gives us transactions, foreign keys, CHECK constraints, and a query planner that optimizes complex joins automatically.

---

12. WHAT IS NOT IMPLEMENTED (AND WHY)

Redis Caching / Distributed Session Store

Why omitted: We achieve "logout everywhere" via database session versioning (O(1) indexed query). We cache AWS Polly voice data in JVM memory (single node, 24h TTL). Redis would add: one more managed service, one more point of failure, one more monthly cost.

When to add:

- **Horizontal scaling:** When we run 3+ Spring Boot nodes, JVM cache becomes inconsistent. Each node caches different voice data. Redis becomes the shared cache.
- **Distributed rate limiting:** When Bucket4j needs a shared Redis store for multi-node quota consistency.
- **Real-time features:** WebSocket session management for async audiobook job notifications.

AWS S3 Audio Storage / CDN Caching

Why omitted: Currently, audio streams directly from Polly to the user. No storage cost.

When to add: Content-addressable caching. If 1,000 users request synthesis of "Hello, welcome to SpeakIT" with voice "Joanna", Polly gets called 1,000 times. With S3 caching:

```
SHA-256("Hello, welcome to SpeakIT" + "Joanna" + "mp3") = "abc123"
Check S3: s3://speakit-audio/abc123.mp3 → EXISTS → serve from Cloudflare CDN (free)
Polly called: 0 times
```

This is the single biggest cost optimization available. At scale, it could reduce Polly costs by 60-80%.

Message Brokers (Kafka/RabbitMQ)

Why omitted: TTS for <3,000 characters completes in <1 second synchronously. No queuing needed.

When to add: "Audiobook Mode" — synthesizing a 100,000-word document (500+ pages). Polly's character limit per request is 3,000. This requires:

1. Splitting text into 33+ chunks
2. Queuing synthesis jobs via Kafka
3. Parallel processing across workers
4. Stitching MP3 files together (FFmpeg)
5. Notifying user via WebSocket when complete

This is a substantial engineering project — worth building only when the use case is validated.

---

13. ADVANCED INTERVIEW EDGE CASES

Q: Your `sessionVersion` check reads from the DB on every request. Isn't this a performance problem?

A: It's a deliberate, managed tradeoff. The query is:

```
SELECT id, session_version, has_natural_voice_access FROM users WHERE id = ?
```

This hits a **primary key index** — PostgreSQL B-tree index lookup is O(log n), typically 1-2 page reads from SSD. With PgBouncer and HikariCP maintaining warm connections, this executes in 1-3ms.

The alternative (pure stateless JWT, no DB check) means a compromised token is valid for up to 24 hours after the user changes their password. For a TTS SaaS with billing, this is unacceptable.

When we scale to 10,000 requests/second and this becomes a DB hotspot, the fix is: add a short-lived (5-minute) Redis cache of `userId → sessionVersion`. A logout invalidates the cache key. The cache handles 99% of reads; the DB handles invalidation.

Q: Your `sessionVersion` increment is an atomic DB update, but isn't there a race condition if two requests hit the filter at the exact same millisecond?

A: No. The `JwtAuthenticationFilter` executes a `SELECT` projection to read the current `sessionVersion`. This is a point-in-time check under PostgreSQL's `READ COMMITTED` isolation level.

Scenario: User is logging out (UPDATE) while a concurrent request is being validated (SELECT).

- If the SELECT sees the OLD version and the JWT also has the old version → request proceeds (correct — logout hadn't committed yet)
- If the SELECT sees the NEW version (post-commit) and the JWT has the old version → 401 (correct — session is invalidated)
- The UPDATE (`SET version = version + 1`) is inherently atomic in PostgreSQL

No inconsistency is possible. The only "race" is whether the validate-SELECT happens before or after the logout-UPDATE commits, both of which produce correct behavior.

Q: You're using `getReferenceById` to avoid a SELECT. What happens if the user was deleted between JWT validation and the history insert?

A: The JWT validation (SELECT from users) happens in the filter. The `tts_history` insert happens asynchronously in the controller. The time gap is ~10-50ms.

If the user is hard-deleted in that window:

1. `getReferenceById` returns an uninitialized Hibernate Proxy — no SELECT yet
2. When the transaction tries to INSERT into `tts_history`, the FK constraint fires
3. PostgreSQL raises: `ERROR: insert or update on table "tts_history" violates foreign key constraint`
4. Hibernate wraps this as `DataIntegrityViolationException`
5. Our `GlobalExceptionHandler` catches it and logs a warning — no stack trace leak, no 500 error to the user

In practice this is impossible in our soft-delete architecture. We never physically DELETE users — we set `is_active = false`. The FK always resolves. The edge case is documented for hard-delete architectures.

Q: Why use `allocationSize = 50`? If the application crashes, don't you lose 49 IDs?

A: Yes. A crash results in a gap in the ID sequence. This is a non-issue because:

1. **Primary keys are not business identifiers.** Order numbers, invoice numbers, ticket IDs — those need continuity. Database PKs are internal pointers. No user or business process depends on PK continuity.

2. **The alternative is worse.** `IDENTITY` strategy forces a DB round-trip per INSERT, disabling JDBC batch inserts. At 1,000 inserts/second, this adds 1,000 unnecessary network round-trips per second.
3. **Gaps are common in production anyway.** Rolled-back transactions also create gaps. Any production PostgreSQL table has ID gaps. This is expected and normal.

The performance gain (98% fewer ID-generation DB calls during bulk inserts) makes this tradeoff trivially worthwhile.

Q: How are you managing AWS credentials in production? Are they in the Docker image?

A: Absolutely not. Storing credentials in a Docker image would mean anyone who pulls the image from Docker Hub has your AWS credentials. This is a critical security failure.

Current approach (Render/Vercel environments):

- Credentials are set as environment variables in Render's dashboard
- Render injects them as OS-level env vars into the container at startup
- Spring Boot reads: `${AWS_ACCESS_KEY_ID}` and `${AWS_SECRET_ACCESS_KEY}`
- The Docker image contains: zero secrets

Next level (if on AWS ECS/EKS): Replace env var credentials with **IAM Roles for Service Accounts (IRSA)**. The pod is assigned an IAM role, and the AWS SDK automatically fetches short-lived credentials from the EC2 metadata service. No static credentials anywhere — not in env vars, not in config files, not in CI secrets.

Q: How did you handle adding `NOT NULL` columns to a table that already had production data?

A: This is a classic production migration challenge. The naive approach (`ALTER TABLE ADD COLUMN foo NOT NULL`) fails immediately if the table has existing rows — PostgreSQL can't satisfy the NOT NULL constraint on rows with no value for the new column.

The safe three-step migration:

```
-- Step 1: Add as NULLABLE (table stays online, app deploys without error)
ALTER TABLE users ADD COLUMN preferred_voice VARCHAR(50);

-- Step 2: Backfill existing rows (before deploying app code that writes this column)
UPDATE users SET preferred_voice = 'Joanna' WHERE preferred_voice IS NULL;

-- Step 3: Add NOT NULL constraint (now safe — no NULLs exist)
ALTER TABLE users ALTER COLUMN preferred_voice SET NOT NULL;
ALTER TABLE users ALTER COLUMN preferred_voice SET DEFAULT 'Joanna';
```

For large tables (millions of rows), Step 2 runs as a background job with batched UPDATEs to avoid long-running transactions that block concurrent reads/writes.

---

14. SYSTEM DESIGN DISCUSSION (SCALING)

"SpeakIT just went viral. Traffic spiked 10,000%. What breaks first?"

Engineer's systematic answer (in order of likelihood):

1. Database Connections (~immediate)

```
10x Render instances × 10 HikariCP connections = 100 PostgreSQL connections
Supabase free tier limit: 60 connections

Result: "HikariCP connection timeout" → HTTP 500 for all requests
```

Fix: Upgrade Supabase plan for higher connection limits. Reduce HikariCP pool size per instance. Switch to Supabase Transaction Pooler (vs Session Pooler) — more aggressive connection multiplexing for stateless queries.

2. Rate Limiting Inconsistency (Minutes)

Horizontal scaling creates inconsistent quota enforcement (each node has its own in-memory bucket). Some users effectively get 3x their quota.

Fix: Migrate Bucket4j to Redis backend (documented migration path).

3. AWS Polly TPS Quota (Minutes to Hours)

Polly has a default Transactions Per Second (TPS) quota per account (e.g., 100 TPS for standard voices). 10,000x traffic easily exceeds this.

```
Polly returns: ThrottlingException
Spring Boot: 429 or 503 to user
```

Fix: Request AWS quota increase (takes 24-48 hours via Support ticket). Implement exponential backoff + jitter retry in `PollyService`.

4. Infrastructure Cost (Ongoing)

1,000 users each generating 3,000-character audio = real AWS Polly cost per request.

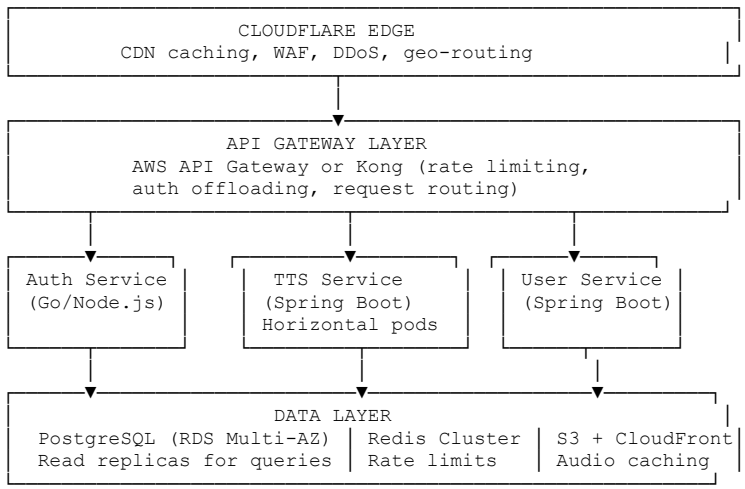
Fix (highest ROI): Implement SHA-256 content-addressable S3 caching. Same text + voice = serve cached MP3 from Cloudflare CDN. Polly cost drops to near-zero for repeated content.

5. Single-Region Latency (Ongoing)

Backend on Render US-East. Users in India/Europe experience 200-400ms additional latency for every API call.

Fix: Migrate to multi-region deployment (Render supports multiple regions). Add Cloudflare Workers for edge-side request routing. For audio streaming, stream through a CDN origin rather than directly from the app server.

"How would you design SpeakIT to handle 1 million daily active users?"



Key changes at 1M DAU:

- **Microservices** — Auth and TTS have very different scaling profiles
- **Redis Cluster** — distributed rate limiting and session caching
- **S3 + Cloudflare** — audio content delivery, eliminating repeat Polly calls
- **Read replicas** — history queries go to replicas; writes go to primary
- **Kafka** — async audiobook generation, decoupled from HTTP request lifecycle
- **Observability** — Datadog/Grafana for distributed tracing, latency percentiles, error rates

---

15. ENGINEERING STORYTELLING

The Founder / CTO Pitch

"We built SpeakIT because high-quality auditory experiences shouldn't require a Hollywood budget. The problem isn't that AWS Polly is hard to use — it's that building a production SaaS around it responsibly requires solving about 20 non-trivial engineering problems: How do you invalidate a JWT instantly without Redis? How do you prevent a single angry user from draining your AWS budget? How

do you stream audio without buffering gigabytes in JVM heap? How do you ensure your character limit isn't bypassed by HTML injection?

We solved all of them. Angular 21 with Signals gives us a reactive, low-latency frontend that serves globally in under 30ms. Spring Boot 3.5 with Java 21 gives us the multi-threaded, strongly-typed backbone needed to stream audio at scale without memory blowups. Our session versioning system invalidates compromised tokens globally with one SQL UPDATE — no Redis, no blacklist, no complexity.

From day one, we built it like something that needs to survive a viral Product Hunt launch: MDC request tracing for instant incident forensics, budget kill switches to prevent runaway AWS costs, multi-stage Docker builds for immutable artifacts, runtime environment injection so staging and prod run identical binaries. It's not over-engineered — every decision has a documented reason and a clear migration path when we outgrow it."

The Interview Story (STAR Format)

Situation: I needed to implement TTS rate limiting in a way that was resistant to proxy rotation, couldn't be bypassed by incognito mode or VPN switching, and was also financially protective (since every Polly call has a cost).

Task: Design and implement an identity-aware, account-bound rate limiting system that works for both anonymous (auth) and authenticated (TTS) endpoints, with graceful client degradation.

Action:

1. Researched rate limiting algorithms — chose Token Bucket over fixed windows because real users naturally burst.
2. Chose Bucket4j over hand-rolling because it provides CAS-based thread safety and nanosecond-precision refill timing (needed for `Retry-After` headers).
3. Implemented `RateLimitAspect` with AOP so rate limiting is a one-annotation addition to any endpoint.
4. Designed `RateLimitAction` enum with different bucket configs per protection zone (auth vs. TTS vs. public API).
5. For unauthenticated endpoints: composite fingerprint (CF-Connecting-IP + User-Agent hash). For authenticated: JWT → User ID directly.
6. Added `Retry-After` header computed from Bucket4j's nanosecond refill ETA, with Angular frontend displaying a live countdown.

Result: Rate limiting that survives VPN rotation and incognito mode, with three distinct protection zones, AOP-based extensibility, and enterprise-grade client feedback. The architecture also doubles as a financial circuit breaker — even if an attacker has valid credentials, they cannot drain our Polly budget beyond the token bucket limit.

Document version: 2.0 | Last updated: June 2025 | Project: github.com/Mohitur669/speakit